In [39]:
```
import sys
sys.executable
```

Out[39]:
```
'c:\\kalpesh\\1.ITvedant\\2.ML projects\\Silage_Sales_ML_Project\\venv\\Scripts\\python.exe'
```

In [41]:
```
import pandas as pd
from sqlalchemy import create_engine
```

In [42]:
```
username = "root"
password = "root"
host = "localhost"
port = "3306"
database = "Pancham_Silage_Factory_DB"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")
```

In [43]:
```
engine.connect()
```

Out[43]:
```
<sqlalchemy.engine.base.Connection at 0x232b1dd7fd0>
```

In [44]:
```
version = sys.version
print(version)
```

```
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
```

# Smart Silage Pricing Model

## 1. Introduction

### Project Objective

The objective of this machine learning project is to analyze and predict the **price per metric ton of silage crops** using historical agricultural trading data.

The model examines key factors such as **crop type, harvest season, moisture content, packaging type, customer segment, and order quantity** to understand how these variables influence pricing and sales outcomes.

By applying machine learning techniques, the system aims to support **data-driven decision making in agricultural supply chains**, helping businesses optimize pricing strategies, manage logistics costs, and improve overall profitability.

# 1. Importing essential python libraries

In [45]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

import warnings
warnings.filterwarnings('ignore')
```

# 2. Dataset Loading

In [46]:
```
df = pd.read_sql("SELECT * FROM main_data", engine)
df.head()
```

Out[46]:

| | Transaction_ID | Date | Customer_Type | Crop_Type | Harvest_Season | Moisture_Content_Percent | DM_Content_Percent | Quantity_M |
|---|---|---|---|---|---|---|---|---|
| **0** | TXN2022-000077 | 2022-07-08 | Co-operative | Maize | Kharif | 69.1 | 30.9 | 9.3 |
| **1** | TXN2022-000092 | 2022-09-28 | Individual Farmer | Bajra | Kharif | 67.4 | 32.6 | 8.2 |
| **2** | TXN2022-000126 | 2022-12-06 | Individual Farmer | Maize | Rabi | 64.8 | 35.2 | 12.1 |
| **3** | TXN2022-000128 | 2022-12-06 | Dairy Farm | Bajra | Rabi | 59.1 | 40.9 | 3.4 |
| **4** | TXN2022-000133 | 2022-06-11 | Individual Farmer | Jowar | Summer | 68.4 | 31.6 | 5.6 |

# 3. Data Cleaning & Preprocessing

In [47]:
```
df.info()
df.isnull().sum()
df.columns = df.columns.str.strip()
df.columns = df.columns.str.title()
df.fillna('0',inplace=True)
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4759 entries, 0 to 4758
Data columns (total 15 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Transaction_ID            4759 non-null   object
 1   Date                      4759 non-null   object
 2   Customer_Type             4759 non-null   object
 3   Crop_Type                 4759 non-null   object
 4   Harvest_Season            4759 non-null   object
 5   Moisture_Content_Percent  4759 non-null   float64
 6   DM_Content_Percent        4759 non-null   float64
 7   Quantity_MT               4759 non-null   float64
 8   Price_per_MT_INR          4759 non-null   float64
 9   Total_Sales_INR           4759 non-null   float64
 10  Logistics_Cost_INR        4759 non-null   float64
 11  Profit_Margin_Percent     4759 non-null   float64
 12  Payment_Mode              4759 non-null   object
 13  Credit_Period_Days        4759 non-null   int64
 14  Bagging_Type              4759 non-null   object
dtypes: float64(7), int64(1), object(7)
memory usage: 557.8+ KB
```

Out[47]:

| | Transaction_Id | Date | Customer_Type | Crop_Type | Harvest_Season | Moisture_Content_Percent | Dm_Content_Percent | Quantity_M |
|---|---|---|---|---|---|---|---|---|
| 0 | TXN2022-000077 | 2022-07-08 | Co-operative | Maize | Kharif | 69.1 | 30.9 | 9.3 |
| 1 | TXN2022-000092 | 2022-09-28 | Individual Farmer | Bajra | Kharif | 67.4 | 32.6 | 8.2 |
| 2 | TXN2022-000126 | 2022-12-06 | Individual Farmer | Maize | Rabi | 64.8 | 35.2 | 12.1 |
| 3 | TXN2022-000128 | 2022-12-06 | Dairy Farm | Bajra | Rabi | 59.1 | 40.9 | 3.4 |
| 4 | TXN2022-000133 | 2022-06-11 | Individual Farmer | Jowar | Summer | 68.4 | 31.6 | 5.6 |

In [48]:
```
df.columns
```

Out[48]:
```
Index(['Transaction_Id', 'Date', 'Customer_Type', 'Crop_Type',
       'Harvest_Season', 'Moisture_Content_Percent', 'Dm_Content_Percent',
       'Quantity_Mt', 'Price_Per_Mt_Inr', 'Total_Sales_Inr',
       'Logistics_Cost_Inr', 'Profit_Margin_Percent', 'Payment_Mode',
       'Credit_Period_Days', 'Bagging_Type'],
      dtype='object')
```

In [49]:
```
df.drop(columns=['Transaction_Id', 'Date', 'Dm_Content_Percent'], inplace=True)
```

In [50]:
```
df.head()
```

Out[50]:

| | Customer_Type | Crop_Type | Harvest_Season | Moisture_Content_Percent | Quantity_Mt | Price_Per_Mt_Inr | Total_Sales_Inr | Logistic |
|---|---|---|---|---|---|---|---|---|
| 0 | Co-operative | Maize | Kharif | 69.1 | 9.37 | 6284.0 | 61682.71 | |
| 1 | Individual Farmer | Bajra | Kharif | 67.4 | 8.23 | 7608.0 | 63157.02 | |
| 2 | Individual Farmer | Maize | Rabi | 64.8 | 12.16 | 6885.0 | 91200.00 | |
| 3 | Dairy Farm | Bajra | Rabi | 59.1 | 3.43 | 6664.0 | 28088.27 | |
| 4 | Individual Farmer | Jowar | Summer | 68.4 | 5.69 | 8161.0 | 51966.77 | |

# 4.Feature Engineering`

In [51]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4759 entries, 0 to 4758
Data columns (total 12 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Customer_Type             4759 non-null   object
 1   Crop_Type                 4759 non-null   object
 2   Harvest_Season            4759 non-null   object
 3   Moisture_Content_Percent  4759 non-null   float64
 4   Quantity_Mt               4759 non-null   float64
 5   Price_Per_Mt_Inr          4759 non-null   float64
 6   Total_Sales_Inr           4759 non-null   float64
 7   Logistics_Cost_Inr        4759 non-null   float64
 8   Profit_Margin_Percent     4759 non-null   float64
 9   Payment_Mode              4759 non-null   object
 10  Credit_Period_Days        4759 non-null   int64
 11  Bagging_Type              4759 non-null   object
dtypes: float64(6), int64(1), object(5)
memory usage: 446.3+ KB
```

In [52]:

```python
def classify_quality(row):

    moisture = row["Moisture_Content_Percent"]

    # Ideal silage ranges
    if 62 <= moisture <= 68:
        return "High Quality Silage"

    else:
        return "Average Quality Silage"


df["Silage_Quality"] = df.apply(classify_quality, axis=1)



def quantity_discount(q):
    if q < 5:
        return 0.025
    elif q < 10:
        return 0.05
    elif q < 25:
        return 0.10
    else:
        return 0.15

df["Discount_rate"] = df["Quantity_Mt"].apply(quantity_discount).astype(float)




df.columns
```

Out[52]:

```
Index(['Customer_Type', 'Crop_Type', 'Harvest_Season',
       'Moisture_Content_Percent', 'Quantity_Mt', 'Price_Per_Mt_Inr',
       'Total_Sales_Inr', 'Logistics_Cost_Inr', 'Profit_Margin_Percent',
       'Payment_Mode', 'Credit_Period_Days', 'Bagging_Type', 'Silage_Quality',
       'Discount_rate'],
      dtype='object')
```

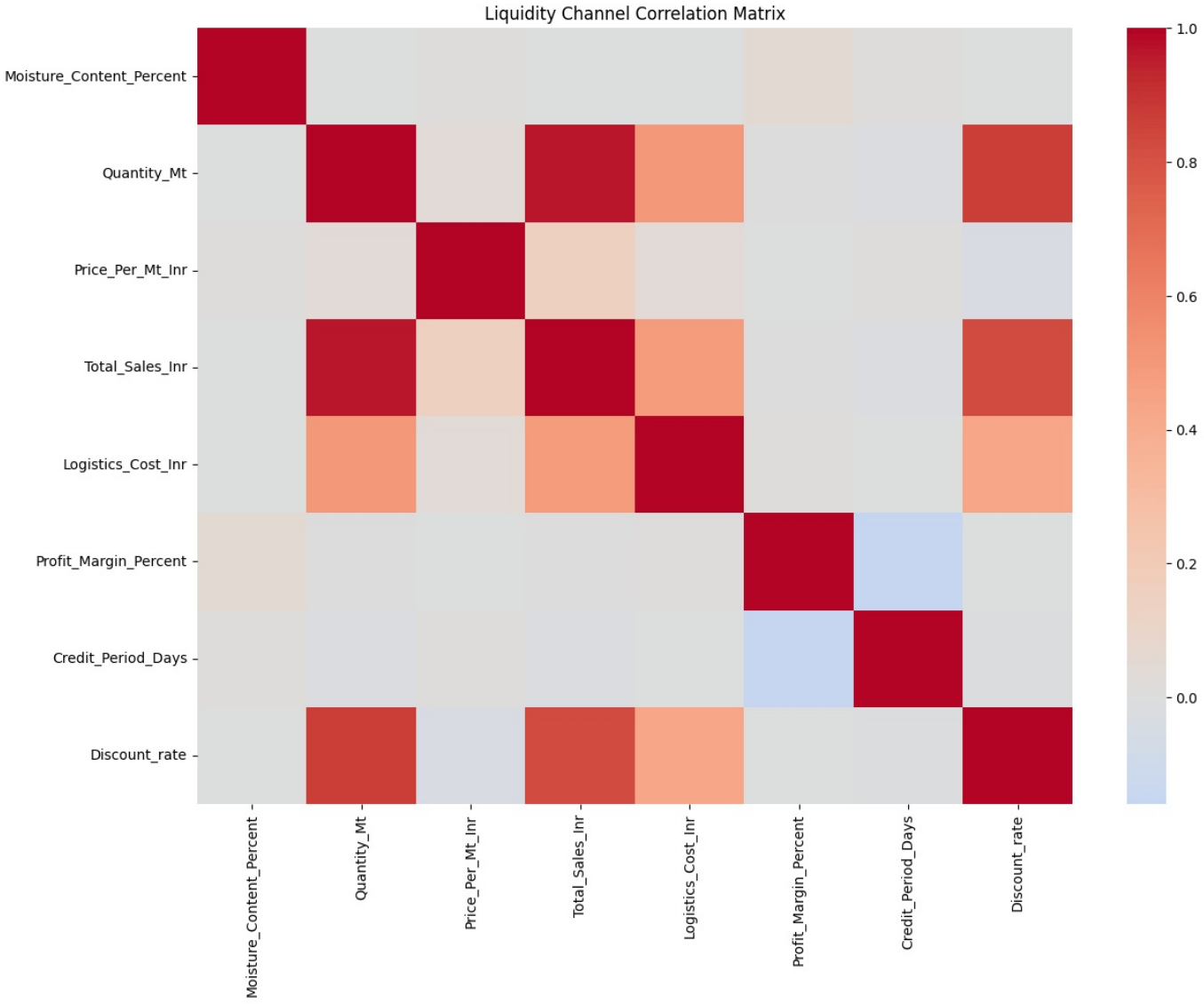
# Feature Selection

In [53]:

```python
import seaborn as sns
import matplotlib.pyplot as plt

corr = df.select_dtypes(include=["int64", "float64"]) \
           .corr()


plt.figure(figsize=(14,10))
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Liquidity Channel Correlation Matrix")
plt.show()
```




In [54]:
```
df.head(2)
```

Out[54]:

| | Customer_Type | Crop_Type | Harvest_Season | Moisture_Content_Percent | Quantity_Mt | Price_Per_Mt_Inr | Total_Sales_Inr | Logistic |
|---|---|---|---|---|---|---|---|---|
| **0** | Co-operative | Maize | Kharif | 69.1 | 9.37 | 6284.0 | 61682.71 | |
| **1** | Individual Farmer | Bajra | Kharif | 67.4 | 8.23 | 7608.0 | 63157.02 | |

In [55]:
```
df.columns
```

Out[55]:
```
Index(['Customer_Type', 'Crop_Type', 'Harvest_Season',
       'Moisture_Content_Percent', 'Quantity_Mt', 'Price_Per_Mt_Inr',
       'Total_Sales_Inr', 'Logistics_Cost_Inr', 'Profit_Margin_Percent',
       'Payment_Mode', 'Credit_Period_Days', 'Bagging_Type', 'Silage_Quality',
       'Discount_rate'],
      dtype='object')
```

In [56]:
```
num_cols = ['Discount_rate']

cat_cols = ['Customer_Type', 'Crop_Type', 'Harvest_Season', 'Bagging_Type', 'Silage_Quality']

target = 'Price_Per_Mt_Inr'
```

In [57]:
```
from sklearn.model_selection import train_test_split

x = df[num_cols + cat_cols]

y = df[target]

xtrain, xtest, ytrain, ytest = train_test_split(x,y,test_size=0.3,random_state=42)
```

In [58]:
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, RobustScaler
from sklearn.impute import SimpleImputer
```

```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", RobustScaler())
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_pipeline, num_cols),
        ("cat", cat_pipeline, cat_cols)
    ]
)
remainder="drop"
```

In [59]:
```
xctrain_processed = preprocessor.fit_transform(xtrain)
xctest_processed = preprocessor.transform(xtest)
```

# BEST REGRESSION MODEL (Predicting the price of silage)

Model Training, Model Evaluation & Model Training

In [60]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate_reg_model(model, xtest, ytest, name):

    # Predictions
    preds = model.predict(xtest)

    # Metrics
    mae = mean_absolute_error(ytest, preds)
    rmse = np.sqrt(mean_squared_error(ytest, preds))
    r2 = r2_score(ytest, preds)

    # Return metrics
    return {
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    }
```

## 1. Linear Regression

In [71]:
```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

lr_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", LinearRegression())
])

best_lr = lr_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_lr, xtest, ytest, "Linear Regression")
```

Out[71]:
```
{'Model': 'Linear Regression',
 'MAE': 88.53781512605042,
 'RMSE': 129.45432232500949,
 'R2': 0.9681646910271192}
```

## 2. Ridge Regression

In [62]:
```
from sklearn.linear_model import Ridge

ridge_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", Ridge(alpha=5.0))
])

best_ridge = ridge_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_ridge, xtest, ytest, "Ridge Regression")
```

Out[62]:
```
{'Model': 'Ridge Regression',
 'MAE': 88.4347425957937,
 'RMSE': 129.14378382847337,
 'R2': 0.9683172426190677}
```

### 3. Lasso Regression

In [63]:
```python
from sklearn.linear_model import Lasso

lasso_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", Lasso(alpha=0.1))
])

best_lasso = lasso_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_lasso, xtest, ytest, "Lasso Regression")
```

Out[63]:
```
{'Model': 'Lasso Regression',
 'MAE': 88.58693300466572,
 'RMSE': 129.41804117044123,
 'R2': 0.9681825329947007}
```

### 4. KNeighbors Regressor

In [64]:
```python
from sklearn.neighbors import KNeighborsRegressor

knn_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", KNeighborsRegressor(
        n_neighbors=10,
        weights="distance"
    ))
])

best_knn = knn_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_knn, xtest, ytest, "KNN Regressor")
```

Out[64]:
```
{'Model': 'KNN Regressor',
 'MAE': 107.5562498807747,
 'RMSE': 158.95413551003412,
 'R2': 0.9520023890812026}
```

### 5. Decision Tree Regressor

In [65]:
```python
from sklearn.tree import DecisionTreeRegressor

dt_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", DecisionTreeRegressor(
        max_depth=10,
        min_samples_split=10,
        random_state=42
    ))
])

best_dt = dt_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_dt, xtest, ytest, "Decision Tree")
```

Out[65]:
```
{'Model': 'Decision Tree',
 'MAE': 95.65392891249272,
 'RMSE': 137.61429533034288,
 'R2': 0.9640248130681324}
```

### 6. Random Forest Regressor

In [66]:
```python
from sklearn.ensemble import RandomForestRegressor

rf_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=200,
        max_depth=20,
        random_state=42,
        n_jobs=-1
    ))
])
```

```
best_rf = rf_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_rf, xtest, ytest, "Random Forest")
```

Out[66]:
```
{'Model': 'Random Forest',
 'MAE': 95.63473757944823,
 'RMSE': 138.9486665814728,
 'R2': 0.9633237668475327}
```

## Gradient Boosting Regressor

In [67]:
```
from sklearn.ensemble import GradientBoostingRegressor

gb_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", GradientBoostingRegressor(
        n_estimators=200,
        learning_rate=0.05,
        random_state=42
    ))
])

best_gb = gb_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_gb, xtest, ytest, "Gradient Boosting")
```

Out[67]:
```
{'Model': 'Gradient Boosting',
 'MAE': 86.37056472804049,
 'RMSE': 127.63605725556623,
 'R2': 0.9690527033226604}
```

## 8. Support Vector Regressor

In [68]:
```
from sklearn.svm import SVR

svr_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", SVR(
        kernel="rbf",
        C=10,
        epsilon=0.1
    ))
])

best_svr = svr_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_svr, xtest, ytest, "SVR")
```

Out[68]:
```
{'Model': 'SVR',
 'MAE': 185.90228666456355,
 'RMSE': 255.81413687713086,
 'R2': 0.8756845690402114}
```

## # 9. XGB Regressor

In [72]:
```
from xgboost import XGBRegressor

xgb_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", XGBRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    ))
])

best_xgb = xgb_pipe.fit(xtrain, ytrain)

evaluate_reg_model(best_xgb, xtest, ytest, "XGBoost")
```

Out[72]:
```
{'Model': 'XGBoost',
 'MAE': 90.11365053834034,
 'RMSE': 132.91417378485124,
 'R2': 0.9664402629008382}
```

In [73]:
```
results = []
```

```
results.append(evaluate_reg_model(best_lr, xtest, ytest, "Linear Regression"))
results.append(evaluate_reg_model(best_ridge, xtest, ytest, "Ridge"))
results.append(evaluate_reg_model(best_lasso, xtest, ytest, "Lasso"))
results.append(evaluate_reg_model(best_knn, xtest, ytest, "KNN"))
results.append(evaluate_reg_model(best_dt, xtest, ytest, "Decision Tree"))
results.append(evaluate_reg_model(best_rf, xtest, ytest, "Random Forest"))
results.append(evaluate_reg_model(best_gb, xtest, ytest, "Gradient Boosting"))
results.append(evaluate_reg_model(best_svr, xtest, ytest, "SVR"))
results.append(evaluate_reg_model(best_xgb, xtest, ytest, "XGBoost"))

results_df = pd.DataFrame(results)

results_df.sort_values("R2", ascending=False)
```

Out[73]:

| | Model | MAE | RMSE | R2 |
|---|---|---|---|---|
| **6** | Gradient Boosting | 86.370565 | 127.636057 | 0.969053 |
| **1** | Ridge | 88.434743 | 129.143784 | 0.968317 |
| **2** | Lasso | 88.586933 | 129.418041 | 0.968183 |
| **0** | Linear Regression | 88.537815 | 129.454322 | 0.968165 |
| **8** | XGBoost | 90.113651 | 132.914174 | 0.966440 |
| **4** | Decision Tree | 95.653929 | 137.614295 | 0.964025 |
| **5** | Random Forest | 95.634738 | 138.948667 | 0.963324 |
| **3** | KNN | 107.556250 | 158.954136 | 0.952002 |
| **7** | SVR | 185.902287 | 255.814137 | 0.875685 |

In [ ]:

```
best_model = best_gb  # or best_log, best_svc, best_gb, etc.
```

In [77]:

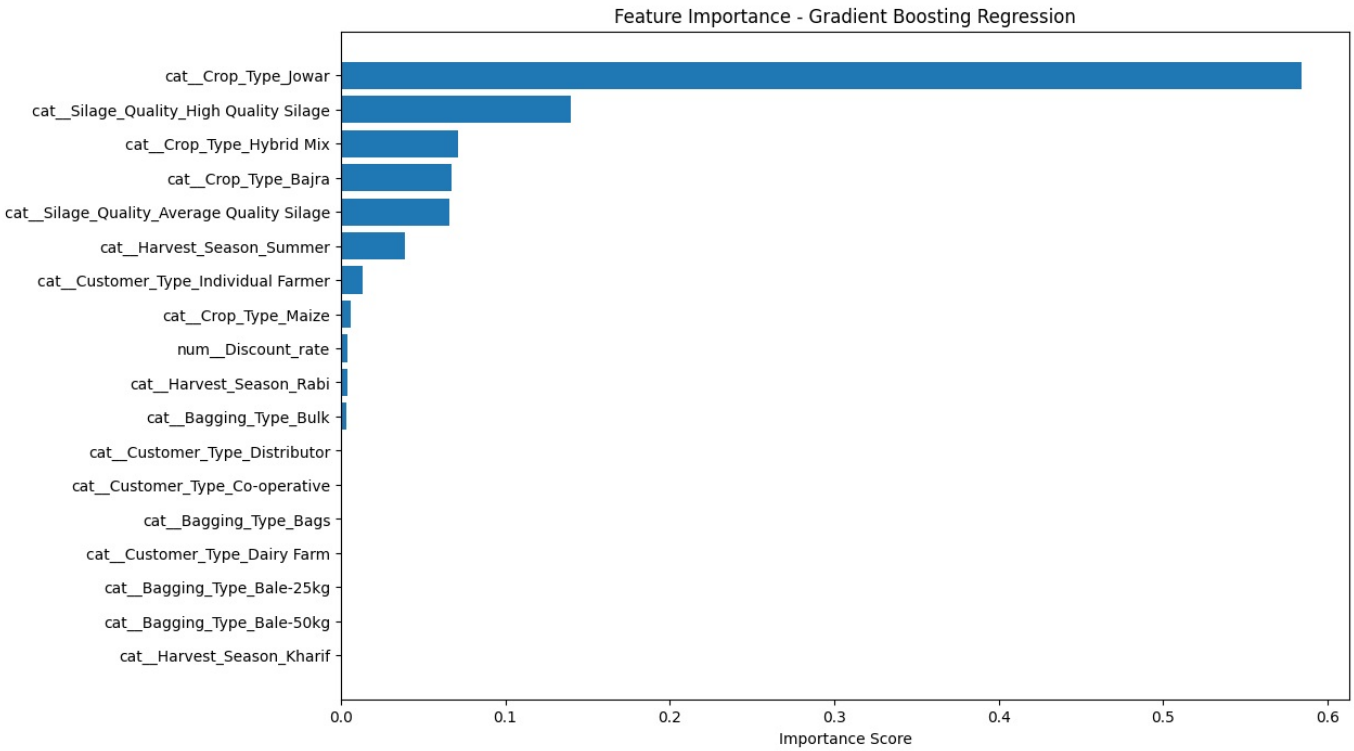
```
# Use Gradient Boosting model
best_model = best_gb

gb_model = best_model.named_steps["model"]
preprocessor = best_model.named_steps["preprocessing"]

importance = gb_model.feature_importances_
feature_names = preprocessor.get_feature_names_out()

indices = np.argsort(importance)

plt.figure(figsize=(12,8))
plt.barh(range(len(indices)), importance[indices])
plt.yticks(range(len(indices)), feature_names[indices])
plt.title("Feature Importance - Gradient Boosting Regression")
plt.xlabel("Importance Score")
plt.show()
```



# Testing

In [ ]:
```
EXPECTED_COLUMNS = xtrain.columns.tolist()

def safe_input(df):
    for col in EXPECTED_COLUMNS:
        if col not in df:
            df[col] = None
    return df[EXPECTED_COLUMNS]
```

In [ ]:
```
import pandas as pd

user_input = pd.DataFrame([{
    "Customer_Type": "Dairy Farm",
    "Crop_Type": "Jowar",
    "Harvest_Season": "Rabi",
    "Moisture_Content_%": 63,
    "DM_Content_%": 37,
    "Quantity_MT": 30,
    "Bagging_Type": "Bags",
    "Quality": "Good"
}])

# Apply same preprocessing as training pipeline
user_input = safe_input(user_input)

prediction = best_model.predict(user_input)

print("Predicted Price per MT:", prediction[0])
```

```
Predicted Price per MT: 7913.8295777282665
```